

AWS Cloud Security Dashboard

Project Deep Dive and Interview Preparation Guide

1. Project Overview

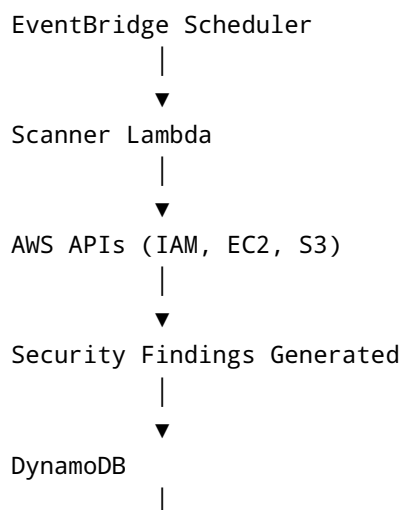
The AWS Cloud Security Dashboard is a serverless cloud security posture management platform designed to continuously monitor AWS resources, detect security misconfigurations, persist findings, and visualize security posture through a web dashboard.

The project demonstrates practical implementation of:

- AWS Serverless Architecture
 - Infrastructure as Code (Terraform)
 - Cloud Security Monitoring
 - Event-driven Architecture
 - REST API Development
 - Cloud Native Dashboarding
-

2. End-to-End Flow

The complete flow of the project is shown below.





3. Detailed Execution Flow

Step 1 — EventBridge Triggers Scanner Lambda

Every day EventBridge triggers the Scanner Lambda.

Terraform Module:

```
terraform/modules/eventbridge
```

Terraform Resource:

```
resource "aws_cloudwatch_event_rule"
```

Example:

```
schedule_expression = "rate(1 day)"
```

This creates a serverless scheduler.

Equivalent Linux Cron:

```
0 0 * * *
```

Interview Question:

Q:

Why did you use EventBridge instead of cron jobs?

Answer:

EventBridge is a fully managed AWS scheduling service. It eliminates server management, scales automatically, integrates natively with Lambda, and follows serverless design principles.

4. Scanner Lambda Deep Dive

Location:

```
backend/
```

Main responsibility:

- Scan AWS resources.
- Detect misconfigurations.
- Generate findings.
- Store findings.
- Send alerts.

Lambda Runtime:

```
Python 3.13
```

Terraform Module:

```
terraform/modules/lambda
```

Terraform Resource:

```
resource "aws_lambda_function" "scanner"
```

Important Configuration:

```
handler = "app.handler"  
  
runtime = "python3.13"
```

```
timeout = 30
```

5. How Does Scanner Lambda Work?

Flow:

```
graph TD
    A[Lambda Invocation] --> B[scanner_runner.py]
    B --> C[Call individual scanners]
    C --> D["S3 Scan  
IAM Scan  
Security Group Scan"]
    D --> E[Generate Findings]
    E --> F[Persist to DynamoDB]
```

6. Security Group Scanner

Purpose:

Detect public SSH exposure.

AWS API Used:

```
ec2.describe_security_groups()
```

Logic:

```
if port == 22 and cidr == "0.0.0.0/0":
```

Finding:

SSH Open To Internet

Severity:

CRITICAL

Reason:

Public SSH access increases attack surface and brute-force risk.

Interview Question:

Q:

Why is opening port 22 to the internet dangerous?

Answer:

Attackers can continuously perform brute force attacks against exposed SSH endpoints. This violates security best practices and increases compromise risk.

7. S3 Scanner

Checks:

Encryption

API:

```
s3.get_bucket_encryption()
```

Finding:

Bucket Encryption Disabled

Severity:

HIGH

Versioning

API:

```
s3.get_bucket_versioning()
```

Finding:

Bucket Versioning Disabled

Severity:

MEDIUM

Reason:

Versioning protects against accidental deletion and ransomware.

Interview Question:

Q:

Why is S3 versioning important?

Answer:

Versioning provides object recovery capabilities, protects against accidental deletion, and helps mitigate ransomware attacks.

8. IAM Scanner

Checks:

Users attached with:

```
AdministratorAccess
```

API:

```
iam.list_attached_user_policies()
```

Finding:

```
Administrator Access Assigned
```

Severity:

```
CRITICAL
```

Reason:

Violates Principle of Least Privilege.

Interview Question:

Q:

Explain least privilege.

Answer:

Users should receive only the minimum permissions required to perform their tasks.

9. DynamoDB Deep Dive

Purpose:

Persist all findings.

Terraform Module:

```
terraform/modules/dynamodb
```

Resource:

```
resource "aws_dynamodb_table"
```

Table Stores:

```
finding_id
title
severity
resource_type
resource_id
created_at
description
```

Why DynamoDB?

- Serverless
- Highly available
- No infrastructure management
- Low cost
- Automatic scaling

Interview Question:

Q:

Why did you choose DynamoDB instead of RDS?

Answer:

The workload is document-oriented, highly scalable, and serverless. No complex joins are required, making DynamoDB more suitable.

10. API Architecture

Frontend never directly accesses DynamoDB.

Architecture:

```
React
|
API Gateway
|
API Lambda
```


|
DynamoDB

Benefits:

- Better security
- Business logic isolation
- Easier maintenance
- Future extensibility

11. API Gateway

Purpose:

Expose REST APIs.

Terraform Module:

```
terraform/modules/api_gateway
```

Resources:

```
aws_api_gateway_rest_api  
aws_api_gateway_method  
aws_api_gateway_integration  
aws_api_gateway_stage
```

API Endpoints:

```
GET /summary  
GET /findings  
GET /history  
GET /lifecycle
```

12. API Lambda Deep Dive

Location:

```
backend/api
```

Purpose:

Serve dashboard requests.

Handler:

```
handler(event, context)
```

Flow:

```
graph TD
    A[Incoming Request] --> B[Read event["path"]]
    B --> C[Route Request]
    C --> D[Call Service Layer]
    D --> E[Return JSON]
```

Example:

```
if path == "/findings":
    return build_response(
        get_all_findings()
    )
```

Equivalent to:

```
@GetMapping("/findings")
```

in Spring Boot.

13. Service Layer Design

Folder:

```
services/
```

Contains:

```
findings_query_service.py  
  
summary_service.py  
  
lifecycle_service.py
```

Purpose:

Separate business logic from API layer.

Benefits:

- Reusability
- Maintainability
- Easier testing

Interview Question:

Q:

Why did you separate business logic into services?

Answer:

Separation of concerns improves maintainability, testability, and code organization.

14. Lifecycle Feature Deep Dive

Purpose:

Track remediation status.

Flow:

```
graph TD
    A[Historical Findings] --> B[Compare With]
    B --> C[Active Findings]
    C --> D[Determine Status]
```

Algorithm:

```
if key in active_keys:
    status = "ACTIVE"
else:
    status = "RESOLVED"
```

Commercial Equivalent:

- Wiz
- Prisma Cloud
- Microsoft Defender for Cloud

Interview Question:

Q:

How does your lifecycle feature work?

Answer:

The system compares all historical findings against currently active findings. Findings still present are marked ACTIVE, while missing findings are marked RESOLVED.

15. Frontend Architecture

Technology:

```
React
TypeScript
Material UI
Axios
Vite
```

Pages:

```
Dashboard.tsx

History.tsx

Lifecycle.tsx
```

Responsibilities:

- Fetch API data.
 - Render charts.
 - Display tables.
 - Present security posture.
-

16. Terraform Architecture

Project Structure:

```
terraform/
|
├── environments/dev
|
└── modules/
```

Why modules?

- Reusability
- Separation of concerns
- Environment portability

Modules:

```
api_gateway
api_lambda
dynamodb
eventbridge
frontend_hosting
iam
lambda
sns
```

17. IAM Role Deep Dive

Single IAM Role:

```
cloud-security-dashboard-dev-scanner-role
```

Permissions:

```
DynamoDB
SNS
EC2 Read
IAM Read
S3 Read
CloudWatch Logs
```

Terraform:

```
aws_iam_role
aws_iam_policy
aws_iam_role_policy_attachment
```

18. Which AWS Account Is Being Monitored?

Current Design:

The application monitors the AWS account in which the Scanner Lambda executes.

Reason:

Lambda automatically receives temporary credentials for its own account through its execution role.

Example:

```
boto3.client("ec2")
```

uses:

```
Lambda Execution Role Credentials
```

automatically.

Therefore:

```
Scanner Lambda Account  
=  
Monitored Account
```

19. How To Change The Monitored Account?

Simply deploy the infrastructure in another AWS account.

Steps:

1. Configure new credentials.

```
aws configure
```

1. Run:

```
terraform apply
```

Infrastructure will deploy into the new account.

Scanner will automatically scan that account.

No code changes required.

20. Can One Account Monitor Another AWS Account?

Current Implementation:

```
NO
```

Reason:

The scanner uses default boto3 credentials from the Lambda execution role.

Those credentials only have permissions inside the current AWS account.

21. How To Implement Cross-Account Monitoring?

AWS Recommended Approach:

Use:

```
STS AssumeRole
```

Architecture:

```
Security Account
|
Lambda
|
STS AssumeRole
|
```


Target Account Role
|
Access Resources

Target Account IAM Role:

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS":
      "arn:aws:iam::<SecurityAccount>:root"
  },
  "Action": "sts:AssumeRole"
}
```

Python:

```
sts.assume_role()
```

After assumption:

```
boto3.Session(
    aws_access_key_id=...
```

This enables centralized multi-account scanning.

Interview Question:

Q:

How would you extend the project for multi-account support?

Answer:

I would create cross-account IAM roles in target accounts and use AWS STS AssumeRole from a central security account.

22. Cost Optimization

Implemented:

- Fully serverless architecture.
 - Pay-per-use services.
 - EventBridge schedule disable option.
 - No EC2 instances.
-

23. Common Interview Questions

Q:

Why serverless?

Q:

Why Terraform?

Q:

Why DynamoDB?

Q:

Why API Gateway?

Q:

Why separate API Lambda and Scanner Lambda?

Q:

How does Lambda scale?

Q:

What is Terraform state?

Q:

What happens if Terraform state is lost?

Q:

How do you debug Lambda failures?

Q:

What causes API Gateway 502 errors?

Q:

How does EventBridge trigger Lambda?

Q:

How do Lambda environment variables work?

Q:

Explain least privilege.

Q:

How would you secure the API?

Q:

How would you implement authentication?

Q:

How would you implement CI/CD?

Q:

How would you support multiple AWS accounts?

24. Potential Future Enhancements

- Cross-account monitoring
- AWS Organizations integration
- Cognito authentication
- CloudFront
- CI/CD pipeline
- Remote Terraform state
- Automated tests

- AWS Config integration
 - CIS Benchmark support
 - CloudTrail analysis
-

25. Key Takeaways

This project demonstrates:

- Cloud Engineering
- DevOps
- Infrastructure as Code
- Serverless Computing
- Cloud Security
- Monitoring and Alerting
- API Development
- Frontend Integration
- Cloud Troubleshooting

The project closely resembles the architecture of modern Cloud Security Posture Management (CSPM) platforms.